Doko Quick-Commerce Platform

Complete Development Guide - Step-by-Step Implementation

Project Overview: Full-stack quick-commerce delivery platform for Nepal with vendor management, rider coordination, real-time order tracking, and multi-payment integration.

Target Completion: 8-12 weeks
Tech Stack: MERN (MongoDB, Express.js, React.js, Node.js)
Document Version: 1.0
Last Updated: February 24, 2026

Table of Contents

1. Project Architecture Overview
2. Development Environment Setup
3. Folder Structure Creation
4. Database Design & Models
5. Backend API Development
6. Frontend Application Development
7. Admin Dashboard Development
8. Vendor Dashboard Development
9. Rider Mobile App Development
10. Payment Integration
11. Real-time Features Implementation
12. Testing Strategy
13. Deployment Guide
14. Appendix: Complete API Reference

---

1. Project Architecture Overview

1.1 System Architecture

Doko follows a **microservices-oriented monolithic architecture** — structured as separate services but deployed together initially for simplicity, with clear boundaries for future microservices migration[1].

Architecture Diagram:

```
┌──────────────
```

```
│ (Database) │
└──────────────────────────┘
```

1.2 Technology Stack Details

Layer	Technology	Version
Frontend	React.js React Router Axios React-Leaflet (Maps)	18.3+ 6.x 1.x 4.x
Backend	Node.js Express.js Mongoose	20.x LTS 4.x 8.x
Database	MongoDB Atlas Redis (Session/Cache)	7.x 7.x
Authentication	JWT bcrypt	9.x 5.x
Payment	Khalti SDK eSewa API	Latest Latest
Real-time	Socket.io	4.x
File Storage	Cloudinary	Latest
Deployment	Vercel (Frontend) Railway/Render (Backend)	- -

Table 1: Complete technology stack for Doko platform

1.3 Core Features Matrix

User Type	Key Features	Priority
Customer	Product browse, Cart, Checkout	P0
	Address management	P0
	Order tracking (real-time)	P1
	Multiple payment methods	P0
	Wishlist, Reviews, Ratings	P1
Vendor	Product CRUD operations	P0
	Order management dashboard	P0
	Revenue analytics	P1
	Inventory tracking	P1
Rider	Order pickup/delivery flow	P0
	Navigation to vendor/customer	P0
	Earnings dashboard	P1
	OTP-based delivery confirmation	P0
Admin	User/Vendor/Rider approval	P0
	Product moderation	P0
	System analytics	P1
	Violation management	P1

Table 2: Feature priority matrix (P0=Must have, P1=Should have)

---

2. Development Environment Setup

2.1 Prerequisites Installation

Step 1: Install Node.js

1. Visit https://nodejs.org/
2. Download Node.js 20.x LTS (Long Term Support)
3. Run installer and verify installation:

```
node --version # Should show v20.x.x
npm --version # Should show 10.x.x
```

Step 2: Install Git

1. Visit https://git-scm.com/

2. Download and install Git for your OS
3. Verify installation:

git --version # Should show git version 2.x

Step 3: Install MongoDB (Local Development)

Option A - MongoDB Community Edition (Local):

Windows (using Chocolatey)

choco install mongodb

macOS (using Homebrew)

brew tap mongodb/brew
brew install mongodb-community

Ubuntu/Debian

sudo apt-get install mongodb

Option B - MongoDB Atlas (Cloud - Recommended):

1. Visit https://www.mongodb.com/cloud/atlas
2. Create free account
3. Create new cluster (M0 Free tier)
4. Get connection string (keep it safe)

Step 4: Install Visual Studio Code

1. Visit https://code.visualstudio.com/
2. Download and install VS Code
3. Install recommended extensions:
    - ESLint
    - Prettier - Code formatter
    - ES7+ React/Redux/React-Native snippets
    - MongoDB for VS Code
    - REST Client

Step 5: Install Postman (API Testing)

1. Visit https://www.postman.com/downloads/
2. Download and install Postman
3. Create free account

2.2 Create GitHub Repository

Navigate to your projects directory

cd ~/Documents/Projects # or your preferred location

Create project directory

mkdir doko-platform
cd doko-platform

Initialize Git

git init

Create .gitignore file

cat > .gitignore << EOF

Dependencies

node_modules/
.pnp
.pnp.js

Environment variables

.env
.env.local
.env.production

Testing

coverage/

Production

build/
dist/

Misc

.DS_Store
.log
npm-debug.log

IDE

.vscode/
.idea/
*.swp
*.swo

MongoDB

data/
EOF

Create README

cat > README.md << EOF

Doko Quick-Commerce Platform

Full-stack delivery platform for Nepal.

Tech Stack

- MongoDB, Express.js, React.js, Node.js
- Socket.io for real-time features
- Khalti & eSewa payment integration

Getting Started

See DEVELOPMENT_GUIDE.pdf for complete setup instructions.
EOF

Initial commit

git add .
git commit -m "Initial commit: Project structure"

Create repository on GitHub (via browser)

Then connect local to remote:

git remote add origin https://github.com/YOUR_USERNAME/doko-platform.git
git branch -M main
git push -u origin main

---

3. Folder Structure Creation

3.1 Complete Directory Structure

Create the following folder structure exactly as shown[2][3]:

```
doko-platform/
├── backend/
│   ├── src/
│   │   ├── config/
│   │   │   ├── database.js
│   │   │   ├── cloudinary.js
│   │   │   └── redis.js
│   │   ├── models/
│   │   │   ├── User.js
│   │   │   ├── Vendor.js
│   │   │   ├── Rider.js
│   │   │   ├── Admin.js
│   │   │   ├── Product.js
│   │   │   ├── Category.js
│   │   │   ├── Cart.js
│   │   │   ├── Order.js
│   │   │   ├── Orders.js
│   │   │   ├── Address.js
│   │   │   ├── Payment.js
│   │   │   ├── Review.js
│   │   │   ├── Employee.js
│   │   │   ├── Event.js
│   │   │   └── Wishlist.js
│   │   ├── controllers/
│   │   │   ├── authController.js
│   │   │   ├── userController.js
│   │   │   ├── vendorController.js
│   │   │   ├── riderController.js
│   │   │   ├── adminController.js
│   │   │   ├── productController.js
│   │   │   ├── cartController.js
│   │   │   ├── orderController.js
│   │   │   ├── paymentController.js
│   │   │   └── reviewController.js
```

```
│ │ ├── routes/
│ │ │ ├── auth.js
│ │ │ ├── user.js
│ │ │ ├── vendor.js
│ │ │ ├── rider.js
│ │ │ ├── admin.js
│ │ │ ├── product.js
│ │ │ ├── cart.js
│ │ │ ├── order.js
│ │ │ ├── payment.js
│ │ │ └── review.js
│ │ ├── middleware/
│ │ │ ├── auth.js
│ │ │ ├── upload.js
│ │ │ ├── errorHandler.js
│ │ │ └── rateLimiter.js
│ │ ├── utils/
│ │ │ ├── sendOTP.js
│ │ │ ├── generateToken.js
│ │ │ ├── validatePhone.js
│ │ │ └── calculateDistance.js
│ │ ├── services/
│ │ │ ├── khalti.js
│ │ │ ├── esewa.js
│ │ │ ├── notification.js
│ │ │ └── socket.js
│ │ └── server.js
│ ├── .env.example
│ ├── package.json
│ └── README.md
├── frontend/
│ ├── public/
│ │ ├── index.html
│ │ ├── favicon.ico
│ │ └── manifest.json
│ ├── src/
│ │ ├── components/
│ │ │ ├── common/
│ │ │ │ ├── Navbar.jsx
```

```
│ │ │ │ ├── Footer.jsx
│ │ │ │ ├── Loader.jsx
│ │ │ │ └── ErrorBoundary.jsx
│ │ │ ├── product/
│ │ │ │ ├── ProductCard.jsx
│ │ │ │ ├── ProductDetail.jsx
│ │ │ │ └── ProductFilter.jsx
│ │ │ ├── cart/
│ │ │ │ ├── CartItem.jsx
│ │ │ │ └── CartSummary.jsx
│ │ │ └── order/
│ │ │ ├── OrderCard.jsx
│ │ │ └── OrderTracking.jsx
│ │ ├── pages/
│ │ │ ├── customer/
│ │ │ │ ├── Home.jsx
│ │ │ │ ├── Products.jsx
│ │ │ │ ├── ProductDetailPage.jsx
│ │ │ │ ├── Cart.jsx
│ │ │ │ ├── Checkout.jsx
│ │ │ │ ├── Orders.jsx
│ │ │ │ ├── Profile.jsx
│ │ │ │ └── Wishlist.jsx
│ │ │ ├── auth/
│ │ │ │ ├── Login.jsx
│ │ │ │ ├── Register.jsx
│ │ │ │ └── OTPVerification.jsx
│ │ │ ├── vendor/
│ │ │ │ ├── VendorDashboard.jsx
│ │ │ │ ├── VendorProducts.jsx
│ │ │ │ ├── VendorOrders.jsx
│ │ │ │ └── VendorProfile.jsx
│ │ │ ├── rider/
│ │ │ │ ├── RiderDashboard.jsx
│ │ │ │ ├── RiderOrders.jsx
│ │ │ │ └── RiderProfile.jsx
│ │ │ └── admin/
│ │ │ ├── AdminDashboard.jsx
│ │ │ ├── UserManagement.jsx
```

```
│   │   │   ├── VendorManagement.jsx
│   │   │   ├── RiderManagement.jsx
│   │   │   ├── ProductManagement.jsx
│   │   │   └── Analytics.jsx
│   │   ├── services/
│   │   │   ├── api.js
│   │   │   ├── auth.js
│   │   │   └── socket.js
│   │   ├── context/
│   │   │   ├── AuthContext.jsx
│   │   │   ├── CartContext.jsx
│   │   │   └── SocketContext.jsx
│   │   ├── hooks/
│   │   │   ├── useAuth.js
│   │   │   ├── useCart.js
│   │   │   └── useOrders.js
│   │   ├── utils/
│   │   │   ├── constants.js
│   │   │   ├── formatters.js
│   │   │   └── validators.js
│   │   ├── styles/
│   │   │   └── global.css
│   │   ├── App.jsx
│   │   └── index.js
│   ├── .env.example
│   ├── package.json
│   └── README.md
├── rider-app/
│   ├── src/
│   │   ├── components/
│   │   ├── screens/
│   │   ├── navigation/
│   │   └── services/
│   ├── package.json
│   └── README.md
├── docs/
│   ├── API_DOCUMENTATION.md
│   ├── DATABASE_SCHEMA.md
│   └── DEPLOYMENT.md
```

```
├── .gitignore
├── README.md
└── DEVELOPMENT_GUIDE.pdf (this document)
```

3.2 Create Folder Structure - Commands

Execute these commands in your terminal:

Navigate to project root

```
cd doko-platform
```

Create backend structure

```
mkdir -p backend/src/{config,models,controllers,routes,middleware,utils,services}
```

Create frontend structure

```
mkdir -p frontend/{public,src/{components/{common,product,cart,order},pages/{customer,auth,vendor,rider,admin},services,context,hooks,utils,styles}}
```

Create rider app structure

```
mkdir -p rider-app/src/{components,screens,navigation,services}
```

Create documentation folder

```
mkdir docs
```

Verify structure

tree -L 3 # On Unix/Mac

or use 'dir' on Windows

---

4. Database Design & Models

4.1 Database Schema Overview

Doko uses MongoDB with Mongoose ODM for schema definition and validation[4][5]. The database consists of 14 core collections with defined relationships.

Entity Relationship Diagram:

```
┌──────────────────┐  ┌──────────────────────┐
┌──────────────────────┐
│ User │────1:N──│ Address │  ┌────│ Product │
│ │ │ │ │ │ │
│ - name │  │ - label │  │  │ - name │
│ - phone │  │ - city │  │  │ - price │
│ - email │  │ - pincode │  │  │ - stock │
└──────────┬───────┘  └──────────────────────┘  │
└──────────┬──────────────────┘
│  │  │
│  │  │ N:1
│ 1:N │  │
│  │  ┌─────────▼──────────┐
┌─────────▼─────────┐  ┌──────────────────────┐  │  │ Category │
│ Cart │  │ Orders │  │  │  │
│  │  │  │  │  │ - name │
│ - items[] │  │ - orderIds[]│  │  │ - type │
└──────────────────┘  │ - total │  │  └──────────────────────┘
└──────────┬──────────────────┘  │
│  │
│ 1:N │ N:1
┌─────────▼──────────┐  │
```

```
│ Order │──────┘
│ │
│ - status │
│ - productId │
│ - quantity │
└──────────┬──────────┘
           │ N:1
           ▼
┌──────────────────┐
│ Rider │
│ │
│ - name │
│ - bikeNo │
└──────────────────┘
```

4.2 Initialize Backend Project

Navigate to backend directory

cd backend

Initialize Node.js project

npm init -y

Install core dependencies

npm install express mongoose dotenv cors helmet morgan
npm install bcryptjs jsonwebtoken express-validator
npm install multer cloudinary socket.io
npm install redis ioredis
npm install axios

Install development dependencies

npm install --save-dev nodemon eslint prettier

Update package.json scripts

Edit `backend/package.json` and add these scripts:

```
{
"name": "doko-backend",
"version": "1.0.0",
"description": "Doko Quick-Commerce Backend API",
"main": "src/server.js",
"scripts": {
"start": "node src/server.js",
"dev": "nodemon src/server.js",
"test": "echo "No tests yet" && exit 0"
},
"keywords": ["quick-commerce", "delivery", "nepal"],
"author": "Your Name",
"license": "MIT"
}
```

4.3 Environment Configuration

Create `backend/.env.example`:

Server Configuration

```
NODE_ENV=development
PORT=5000
API_VERSION=v1
```

Database

MONGODB_URI=mongodb://localhost:27017/doko

or MongoDB Atlas:

MONGODB_URI=mongodb+srv://username:password@cluster.mongodb.net/doko

JWT Secret

JWT_SECRET=your-super-secret-jwt-key-change-this-in-production
JWT_EXPIRE=7d

Redis (for caching and sessions)

REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_PASSWORD=

Cloudinary (Image Storage)

CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret

SMS/OTP Service (Nepal SMS providers)

SMS_API_KEY=your_sms_api_key
SMS_SENDER_ID=DOKO

Payment Gateways

KHALTI_SECRET_KEY=your_khalti_secret_key
KHALTI_PUBLIC_KEY=your_khalti_public_key
ESEWA_MERCHANT_ID=your_esewa_merchant_id
ESEWA_SECRET_KEY=your_esewa_secret_key

Google Maps API

GOOGLE_MAPS_API_KEY=your_google_maps_api_key

Frontend URL (CORS)

FRONTEND_URL=http://localhost:3000

Admin Credentials (Initial Setup)

ADMIN_EMAIL=admin@doko.com
ADMIN_PHONE=9800000000
ADMIN_PASSWORD=Admin@123

Copy to actual .env file:
cp .env.example .env

Then edit .env with your actual credentials

4.4 Database Connection Setup

Create backend/src/config/database.js:

const mongoose = require('mongoose');

const connectDB = async () => {
try {
const conn = await mongoose.connect(process.env.MONGODB_URI, {
useNewUrlParser: true,
useUnifiedTopology: true,
});

```
console.log(`✅ MongoDB Connected: ${conn.connection.host}`);

// Set up connection event listeners
mongoose.connection.on('error', (err) => {
  console.error(`❌ MongoDB connection error: ${err}`);
});

mongoose.connection.on('disconnected', () => {
  console.warn('⚠️ MongoDB disconnected. Attempting to reconnect...');
});

mongoose.connection.on('reconnected', () => {
  console.log('✅ MongoDB reconnected');
});
```

} catch (error) {
console.error(❌ Error connecting to MongoDB: ${error.message});
process.exit(1);
}
};

```
module.exports = connectDB;
```

4.5 Mongoose Models Implementation

Model 1: User Model

Create `backend/src/models/User.js`:

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
name: {
type: String,
required: [true, 'Please provide a name'],
trim: true,
maxlength: [100, 'Name cannot exceed 100 characters']
},
phone: {
type: String,
required: [true, 'Please provide a phone number'],
unique: true,
match: [/[1]{10}/,
'Please provide a valid email'
]
},
passwordHash: {
type: String,
required: [true, 'Please provide a password'],
minlength: 6,
select: false // Don't return password in queries by default
},
otp: {
code: String,
expiresAt: Date
},
suspended: {
type: Boolean,
default: false
},
deactivated: {
```

```
type: Boolean,
default: false
},
violations: [{
reason: String,
date: {
type: Date,
default: Date.now
},
severity: {
type: String,
enum: ['low', 'medium', 'high'],
default: 'low'
}
}],
defaultAddress: {
type: mongoose.Schema.Types.ObjectId,
ref: 'Address'
}
}, {
timestamps: true
});

// Hash password before saving
userSchema.pre('save', async function(next) {
if (!this.isModified('passwordHash')) {
return next();
}

const salt = await bcrypt.genSalt(10);
this.passwordHash = await bcrypt.hash(this.passwordHash, salt);
next();
});

// Method to compare password
userSchema.methods.comparePassword = async
function(candidatePassword) {
return await bcrypt.compare(candidatePassword,
this.passwordHash);
};
```

```
// Method to generate OTP
userSchema.methods.generateOTP = function() {
const otp = Math.floor(100000 + Math.random() * 900000).toString();
this.otp = {
code: otp,
expiresAt: Date.now() + 10 * 60 * 1000 // 10 minutes
};
return otp;
};

// Method to verify OTP
userSchema.methods.verifyOTP = function(candidateOTP) {
if (!this.otp || !this.otp.code) return false;
if (Date.now() > this.otp.expiresAt) return false;
return this.otp.code === candidateOTP;
};

module.exports = mongoose.model('User', userSchema);
```

Model 2: Product Model

Create backend/src/models/Product.js:

```
const mongoose = require('mongoose');

const productSchema = new mongoose.Schema({
name: {
type: String,
required: [true, 'Product name is required'],
trim: true,
maxlength: [200, 'Product name cannot exceed 200 characters']
},
brand: {
type: String,
required: [true, 'Brand name is required'],
trim: true
},
shortDescription: {
type: String,
required: true,
maxlength: [300, 'Short description cannot exceed 300 characters']
```

```
},
description: {
type: String,
required: true,
maxlength: [2000, 'Description cannot exceed 2000 characters']
},
pricePerUnit: {
type: Number,
required: [true, 'Price is required'],
min: [0, 'Price cannot be negative']
},
unit: {
type: String,
required: true,
enum: ['kg', 'g', 'L', 'ml', 'pcs', 'dozen', 'pack']
},
discount: {
type: Number,
default: 0,
min: [0, 'Discount cannot be negative'],
max: [100, 'Discount cannot exceed 100%']
},
productCategory: {
type: mongoose.Schema.Types.ObjectId,
ref: 'Category',
required: true
},
deliveryCategory: {
type: String,
enum: ['quick', 'ecommerce'],
required: true
},
stock: {
type: Number,
required: true,
min: [0, 'Stock cannot be negative'],
default: 0
},
photos: [{
```

```
type: String,
required: true
}],
vendorId: {
type: mongoose.Schema.Types.ObjectId,
ref: 'Vendor',
required: true
},
rating: {
average: {
type: Number,
default: 0,
min: 0,
max: 5
},
count: {
type: Number,
default: 0
}
},
approved: {
type: Boolean,
default: false
},
hidden: {
type: Boolean,
default: false
},
tags: [String]
}, {
timestamps: true
});

// Index for faster searches
productSchema.index({ name: 'text', brand: 'text', description: 'text' });
productSchema.index({ productCategory: 1, deliveryCategory: 1 });
productSchema.index({ vendorId: 1 });
productSchema.index({ approved: 1, hidden: 1 });
```

```
// Virtual for final price after discount
productSchema.virtual('finalPrice').get(function() {
return this.pricePerUnit - (this.pricePerUnit * this.discount / 100);
});

// Method to check if product is in stock
productSchema.methods.isInStock = function(quantity = 1) {
return this.stock >= quantity;
};

// Method to reduce stock
productSchema.methods.reduceStock = async function(quantity) {
if (!this.isInStock(quantity)) {
throw new Error('Insufficient stock');
}
this.stock -= quantity;
await this.save();
};

module.exports = mongoose.model('Product', productSchema);
```

Model 3: Category Model

Create backend/src/models/Category.js:

```
const mongoose = require('mongoose');

const criteriaSchema = new mongoose.Schema({
name: {
type: String,
required: true
},
description: {
type: String,
required: true
}
}, { _id: false });

const categorySchema = new mongoose.Schema({
name: {
type: String,
required: [true, 'Category name is required'],
```

```
unique: true,
trim: true
},
type: {
type: String,
enum: ['quick', '1day'],
required: true
},
required: [criteriaSchema],
others: [criteriaSchema],
icon: String,
isActive: {
type: Boolean,
default: true
}
}, {
timestamps: true
});

module.exports = mongoose.model('Category', categorySchema);
```

Model 4: Address Model

Create backend/src/models/Address.js:

```
const mongoose = require('mongoose');

const addressSchema = new mongoose.Schema({
userId: {
type: mongoose.Schema.Types.ObjectId,
ref: 'User',
required: true
},
label: {
type: String,
enum: ['Home', 'Work', 'Office', 'Other'],
default: 'Home'
},
name: String,
phone: {
type: String,
```

```
required: true,
match: [/[2]{10}/, 'Please provide a valid pincode']
},
location: {
type: {
type: String,
enum: ['Point'],
default: 'Point'
},
coordinates: {
type: [Number], // [longitude, latitude]
required: true
}
},
isDefault: {
type: Boolean,
default: false
}
}, {
timestamps: true
});

// Geospatial index for location-based queries
addressSchema.index({ location: '2dsphere' });

// Ensure only one default address per user
addressSchema.pre('save', async function(next) {
if (this.isDefault) {
await this.constructor.updateMany(
{ userId: this.userId, _id: { $ne: this._id } },
{ isDefault: false }
);
}
next();
});

module.exports = mongoose.model('Address', addressSchema);
```

Model 5: Vendor Model

Create backend/src/models/Vendor.js:

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const vendorSchema = new mongoose.Schema({
name: {
type: String,
required: true,
trim: true
},
phone: {
type: String,
required: true,
unique: true,
match: [/[3]{10}/, 'Invalid PAN format']
},
storeName: {
type: String,
required: true,
trim: true
},
address: {
type: String,
required: true
},
geolocation: {
type: {
type: String,
enum: ['Point'],
default: 'Point'
},
coordinates: {
type: [Number],
required: true
}
},
businessType: {
type: String,
required: true
},
description: {
```

```
type: String,
maxlength: 1000
},
passwordHash: {
type: String,
required: true,
select: false
},
otp: {
code: String,
expiresAt: Date
},
verified: {
type: Boolean,
default: false
},
suspended: {
type: Boolean,
default: false
},
revenue: {
type: Number,
default: 0
},
violations: [{
reason: String,
date: Date,
severity: {
type: String,
enum: ['low', 'medium', 'high']
}
}],
documents: {
businessLicense: String,
panCard: String,
registrationCertificate: String
}
}, {
```

```
timestamps: true
});

vendorSchema.index({ geolocation: '2dsphere' });

vendorSchema.pre('save', async function(next) {
if (!this.isModified('passwordHash')) return next();
const salt = await bcrypt.genSalt(10);
this.passwordHash = await bcrypt.hash(this.passwordHash, salt);
next();
});

vendorSchema.methods.comparePassword = async
function(password) {
return await bcrypt.compare(password, this.passwordHash);
};

module.exports = mongoose.model('Vendor', vendorSchema);
```

Model 6: Rider Model

Create backend/src/models/Rider.js:

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const riderSchema = new mongoose.Schema({
name: {
type: String,
required: true,
trim: true
},
phone: {
type: String,
required: true,
unique: true
},
email: {
type: String,
required: true,
unique: true,
lowercase: true
```

```
},
passwordHash: {
type: String,
required: true,
select: false
},
otp: {
code: String,
expiresAt: Date
},
address: String,
panCard: String,
citizenship: String,
rcBook: String,
bikeDetails: {
model: String,
number: String,
color: String,
type: {
type: String,
enum: ['bike', 'scooter']
}
},
bikeInsurancePaper: String,
verified: {
type: Boolean,
default: false
},
suspended: {
type: Boolean,
default: false
},
rating: {
average: {
type: Number,
default: 0,
min: 0,
max: 5
},
```

```
count: {
type: Number,
default: 0
}
},
revenue: {
type: Number,
default: 0
},
violations: [{
reason: String,
date: Date,
severity: String
}],
isAvailable: {
type: Boolean,
default: true
},
currentLocation: {
type: {
type: String,
enum: ['Point'],
default: 'Point'
},
coordinates: [Number]
}
}, {
timestamps: true
});

riderSchema.index({ currentLocation: '2dsphere' });

riderSchema.pre('save', async function(next) {
if (!this.isModified('passwordHash')) return next();
const salt = await bcrypt.genSalt(10);
this.passwordHash = await bcrypt.hash(this.passwordHash, salt);
next();
});
```

```
riderSchema.methods.comparePassword = async function(password)
{
return await bcrypt.compare(password, this.passwordHash);
};

module.exports = mongoose.model('Rider', riderSchema);
```

Model 7: Cart Model

Create backend/src/models/Cart.js:

```
const mongoose = require('mongoose');

const cartSchema = new mongoose.Schema({
userId: {
type: mongoose.Schema.Types.ObjectId,
ref: 'User',
required: true
},
items: [{
productId: {
type: mongoose.Schema.Types.ObjectId,
ref: 'Product',
required: true
},
quantity: {
type: Number,
required: true,
min: 1,
default: 1
},
type: {
type: String,
enum: ['grocery', 'ecommerce'],
required: true
},
addedAt: {
type: Date,
default: Date.now
}
}]
```

```
}, {
timestamps: true
});

// Index for faster user cart lookups
cartSchema.index({ userId: 1 });

// Method to calculate cart total
cartSchema.methods.calculateTotal = async function() {
await this.populate('items.productId');

let total = 0;
for (const item of this.items) {
const product = item.productId;
const finalPrice = product.pricePerUnit - (product.pricePerUnit *
product.discount / 100);
total += finalPrice * item.quantity;
}

return total;
};

module.exports = mongoose.model('Cart', cartSchema);
```

Model 8: Order Model

Create backend/src/models/Order.js:

```
const mongoose = require('mongoose');

const orderSchema = new mongoose.Schema({
orderId: {
type: String,
unique: true,
required: true
},
userId: {
type: mongoose.Schema.Types.ObjectId,
ref: 'User',
required: true
},
vendorId: {
```

```
type: mongoose.Schema.Types.ObjectId,
ref: 'Vendor',
required: true
},
riderId: {
type: mongoose.Schema.Types.ObjectId,
ref: 'Rider'
},
productId: {
type: mongoose.Schema.Types.ObjectId,
ref: 'Product',
required: true
},
quantity: {
type: Number,
required: true,
min: 1
},
priceAtOrder: {
type: Number,
required: true
},
status: {
type: String,
enum: ['preparing', 'pending', 'delivered', 'cancelled', 'returned'],
default: 'preparing'
},
type: {
type: String,
enum: ['grocery', 'ecommerce'],
required: true
},
dateOfOrder: {
type: Date,
default: Date.now
},
dateOfDelivery: Date,
deliveryAddress: {
type: mongoose.Schema.Types.ObjectId,
```

```
ref: 'Address',
required: true
},
otp: {
code: String,
verified: {
type: Boolean,
default: false
}
}
}, {
timestamps: true
});

// Indexes for efficient queries
orderSchema.index({ userId: 1, status: 1 });
orderSchema.index({ vendorId: 1, status: 1 });
orderSchema.index({ riderId: 1, status: 1 });
orderSchema.index({ orderId: 1 });

// Generate unique order ID
orderSchema.pre('save', async function(next) {
if (this.isNew) {
const timestamp = Date.now().toString(36);
const random = Math.random().toString(36).substring(2, 7);
this.orderId = DOKO-${timestamp}-${random}.toUpperCase();
}
next();
});

module.exports = mongoose.model('Order', orderSchema);
```

Model 9: Orders (Basket) Model

Create backend/src/models/Orders.js:

```
const mongoose = require('mongoose');

const ordersSchema = new mongoose.Schema({
userId: {
type: mongoose.Schema.Types.ObjectId,
```

```
ref: 'User',
required: true
},
orderIds: [{
type: mongoose.Schema.Types.ObjectId,
ref: 'Order'
}],
subtotal: {
type: Number,
required: true
},
deliveryCharge: {
type: Number,
default: 0
},
tax: {
type: Number,
default: 0
},
total: {
type: Number,
required: true
},
paymentMethod: {
type: String,
enum: ['khalti', 'esewa', 'fonepay', 'cashOnDelivery'],
required: true
},
paymentStatus: {
type: String,
enum: ['pending', 'completed', 'failed'],
default: 'pending'
},
otp: {
code: String,
expiresAt: Date
}
}, {
```

```
timestamps: true
});

ordersSchema.index({ userId: 1 });

module.exports = mongoose.model('Orders', ordersSchema);
```

Model 10: Payment Model

Create backend/src/models/Payment.js:

```
const mongoose = require('mongoose');

const paymentSchema = new mongoose.Schema({
orderId: {
type: mongoose.Schema.Types.ObjectId,
ref: 'Orders',
required: true
},
userId: {
type: mongoose.Schema.Types.ObjectId,
ref: 'User',
required: true
},
amount: {
type: Number,
required: true
},
type: {
type: String,
enum: ['khalti', 'esewa', 'fonepay', 'cashOnDelivery'],
required: true
},
transactionId: String,
status: {
type: String,
enum: ['initiated', 'pending', 'success', 'failed'],
default: 'initiated'
},
gatewayResponse: mongoose.Schema.Types.Mixed
}, {
```

```
timestamps: true
});

paymentSchema.index({ orderId: 1 });
paymentSchema.index({ userId: 1 });

module.exports = mongoose.model('Payment', paymentSchema);
```

Model 11: Review Model

Create `backend/src/models/Review.js`:

```
const mongoose = require('mongoose');

const reviewSchema = new mongoose.Schema({
userId: {
type: mongoose.Schema.Types.ObjectId,
ref: 'User',
required: true
},
productId: {
type: mongoose.Schema.Types.ObjectId,
ref: 'Product',
required: true
},
rating: {
type: Number,
required: true,
min: 1,
max: 5
},
description: {
type: String,
maxlength: 1000
},
images: [String],
verified: {
type: Boolean,
default: false
}
}, {
```

```
timestamps: true
});

reviewSchema.index({ productId: 1 });
reviewSchema.index({ userId: 1, productId: 1 }, { unique: true });

module.exports = mongoose.model('Review', reviewSchema);
```

Model 12: Wishlist Model

Create backend/src/models/Wishlist.js:

```
const mongoose = require('mongoose');

const wishlistSchema = new mongoose.Schema({
userId: {
type: mongoose.Schema.Types.ObjectId,
ref: 'User',
required: true
},
products: [{
productId: {
type: mongoose.Schema.Types.ObjectId,
ref: 'Product'
},
addedAt: {
type: Date,
default: Date.now
}
}]
}, {
timestamps: true
});

wishlistSchema.index({ userId: 1 });

module.exports = mongoose.model('Wishlist', wishlistSchema);
```

Model 13: Admin Model

Create backend/src/models/Admin.js:

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const adminSchema = new mongoose.Schema({
name: {
type: String,
required: true
},
phone: {
type: String,
required: true,
unique: true
},
email: {
type: String,
required: true,
unique: true
},
passwordHash: {
type: String,
required: true,
select: false
},
otp: {
code: String,
expiresAt: Date
},
role: {
type: String,
enum: ['superadmin', 'admin', 'moderator'],
default: 'admin'
},
certificates: {
companyRegistration: String,
panVat: String,
businessLicense: String,
ecommerceListing: String,
moaAoaCitizenship: String,
employeePan: String,
vendorPan: String,
```

```
sharePaper: String,
aomMom: String
},
permissions: {
canApproveVendors: {
type: Boolean,
default: true
},
canApproveRiders: {
type: Boolean,
default: true
},
canModerateProducts: {
type: Boolean,
default: true
},
canManageUsers: {
type: Boolean,
default: true
}
}
}, {
timestamps: true
});

adminSchema.pre('save', async function(next) {
if (!this.isModified('passwordHash')) return next();
const salt = await bcrypt.genSalt(10);
this.passwordHash = await bcrypt.hash(this.passwordHash, salt);
next();
});

adminSchema.methods.comparePassword = async
function(password) {
return await bcrypt.compare(password, this.passwordHash);
};

module.exports = mongoose.model('Admin', adminSchema);
```

Model 14: Employee Model

Create backend/src/models/Employee.js:

```
const mongoose = require('mongoose');

const employeeSchema = new mongoose.Schema({
empId: {
type: String,
required: true,
unique: true
},
empCategory: {
type: String,
required: true
},
position: {
type: String,
required: true
},
name: {
type: String,
required: true
},
phone: {
type: String,
required: true
},
email: {
type: String,
required: true
},
address: String,
citizenship: String,
pan: {
type: String,
required: true
},
salary: {
type: Number,
required: true
},
```

```
ctc: Number,
bankDetails: {
bankName: String,
accountNumber: String,
ifscCode: String
},
documents: {
rpp: String,
educationalCertificate: String
},
violations: [{
reason: String,
date: Date,
severity: String
}],
isActive: {
type: Boolean,
default: true
}
}, {
timestamps: true
});

module.exports = mongoose.model('Employee', employeeSchema);
```

Model 15: Event Model

Create backend/src/models/Event.js:

```
const mongoose = require('mongoose');

const eventSchema = new mongoose.Schema({
type: {
type: String,
enum: [
'order_preparing',
'order_pending',
'order_delivered',
'order_cancelled',
'order_returned',
'payment_khalti',
```

```
'payment_esewa',
'payment_fonepay',
'payment_cod'
],
required: true
},
orderId: {
type: mongoose.Schema.Types.ObjectId,
ref: 'Order',
required: true
},
userId: {
type: mongoose.Schema.Types.ObjectId,
ref: 'User'
},
riderId: {
type: mongoose.Schema.Types.ObjectId,
ref: 'Rider'
},
vendorId: {
type: mongoose.Schema.Types.ObjectId,
ref: 'Vendor'
},
status: {
type: String,
enum: ['running', 'completed'],
default: 'running'
},
metadata: mongoose.Schema.Types.Mixed,
completedAt: Date
}, {
timestamps: true
});

eventSchema.index({ orderId: 1, type: 1 });
eventSchema.index({ status: 1, createdAt: -1 });

module.exports = mongoose.model('Event', eventSchema);
```

4.6 Model Summary & Relationships

Model	Key Relationships	Purpose
User	Address (1:N), Cart (1:1), Orders (1:N)	Customer accounts
Vendor	Product (1:N), Order (1:N)	Store management
Rider	Order (1:N)	Delivery personnel
Product	Category (N:1), Vendor (N:1)	Catalog items
Category	Product (1:N)	Grouping products
Cart	User (1:1), Product (N:N)	Shopping cart
Order	User/Vendor/Rider (N:1)	Individual order
Orders	Order (1:N)	Order basket
Payment	Orders (1:1)	Payment tracking
Address	User (N:1)	Delivery locations
Review	User/Product (N:1)	Feedback system
Wishlist	User (1:1), Product (N:N)	Saved items
Admin	-	Platform management
Employee	-	Staff records
Event	Order (N:1)	Activity logging

Table 3: Database models relationship summary

---

5. Backend API Development

5.1 Server Setup

Create `backend/src/server.js`:

```
const express = require('express');
const dotenv = require('dotenv');
const cors = require('cors');
const helmet = require('helmet');
const morgan = require('morgan');
const connectDB = require('./config/database');
```

```
// Load environment variables
dotenv.config();

// Initialize Express app
const app = express();

// Connect to database
connectDB();

// Middleware
app.use(helmet()); // Security headers
app.use(cors({
origin: process.env.FRONTEND_URL || 'http://localhost:3000',
credentials: true
}));
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(morgan('dev')); // Request logging

// API Routes
app.use('/api/v1/auth', require('./routes/auth'));
app.use('/api/v1/user', require('./routes/user'));
app.use('/api/v1/vendor', require('./routes/vendor'));
app.use('/api/v1/rider', require('./routes/rider'));
app.use('/api/v1/admin', require('./routes/admin'));
app.use('/api/v1/product', require('./routes/product'));
app.use('/api/v1/cart', require('./routes/cart'));
app.use('/api/v1/order', require('./routes/order'));
app.use('/api/v1/payment', require('./routes/payment'));
app.use('/api/v1/review', require('./routes/review'));

// Health check route
app.get('/health', (req, res) => {
res.json({
success: true,
message: 'Doko API is running',
timestamp: new Date().toISOString()
});
});
```

```
// Error handling middleware
app.use(require('./middleware/errorHandler'));

// 404 handler
app.use((req, res) => {
res.status(404).json({
success: false,
message: 'Route not found'
});
});

// Start server
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
console.log(
╔═══════════════════════════════════════════════════════╗ ║ 
Doko API Server Running ║ ║ Port: ${PORT} ║ ║ Environment:
${process.env.NODE_ENV || 'development'} ║ ║ URL:
http://localhost:${PORT} ║
╚═══════════════════════════════════════════════════════╝ );
});

// Handle unhandled promise rejections
process.on('unhandledRejection', (err) => {
console.error(✗ Unhandled Rejection: ${err.message});
process.exit(1);
});
```

5.2 Authentication Middleware

Create `backend/src/middleware/auth.js`:

```
const jwt = require('jsonwebtoken');
const User = require('../models/User');
const Vendor = require('../models/Vendor');
const Rider = require('../models/Rider');
const Admin = require('../models/Admin');

// Protect routes - general authentication
exports.protect = async (req, res, next) => {
let token;
```

```
// Check for token in Authorization header
if (req.headers.authorization &&
req.headers.authorization.startsWith('Bearer')) {
token = req.headers.authorization.split(' ')[1];
}

if (!token) {
return res.status(401).json({
success: false,
message: 'Not authorized to access this route'
});
}

try {
// Verify token
const decoded = jwt.verify(token, process.env.JWT_SECRET);
```

```
// Find user by ID and role
let user;
switch (decoded.role) {
  case 'user':
    user = await User.findById(decoded.id);
    break;
  case 'vendor':
    user = await Vendor.findById(decoded.id);
    break;
  case 'rider':
    user = await Rider.findById(decoded.id);
    break;
  case 'admin':
    user = await Admin.findById(decoded.id);
    break;
  default:
    return res.status(401).json({
      success: false,
      message: 'Invalid user role'
    });
}
```

```
  if (!user) {
    return res.status(404).json({
      success: false,
      message: 'User not found'
    });
  }

  // Check if suspended
  if (user.suspended) {
    return res.status(403).json({
      success: false,
      message: 'Account has been suspended',
      violations: user.violations
    });
  }

  // Attach user to request
  req.user = user;
  req.userRole = decoded.role;
  next();
```

} catch (error) {
return res.status(401).json({
success: false,
message: 'Token has expired or is invalid'
});
}
};

// Role-specific middleware
exports.isUser = (req, res, next) => {
if (req.userRole !== 'user') {
return res.status(403).json({
success: false,
message: 'Access denied. User role required.'
});
}

```
next();
};

exports.isVendor = (req, res, next) => {
if (req.userRole !== 'vendor') {
return res.status(403).json({
success: false,
message: 'Access denied. Vendor role required.'
});
}
next();
};

exports.isRider = (req, res, next) => {
if (req.userRole !== 'rider') {
return res.status(403).json({
success: false,
message: 'Access denied. Rider role required.'
});
}
next();
};

exports.isAdmin = (req, res, next) => {
if (req.userRole !== 'admin') {
return res.status(403).json({
success: false,
message: 'Access denied. Admin role required.'
});
}
next();
};
```

5.3 Utility Functions

JWT Token Generator

Create `backend/src/utils/generateToken.js`:

```
const jwt = require('jsonwebtoken');
```

```
const generateToken = (id, role) => {
return jwt.sign(
{ id, role },
process.env.JWT_SECRET,
{ expiresIn: process.env.JWT_EXPIRE || '7d' }
);
};

module.exports = generateToken;
```

OTP Sender

Create `backend/src/utils/sendOTP.js`:

```
const axios = require('axios');

const sendOTP = async (phone, otp) => {
try {
// Example using Sparrow SMS (Nepal)
const response = await axios.post('
https://sms.aakashsms.com/sms/v3/send', {
auth_token: process.env.SMS_API_KEY,
to: phone,
text: Your Doko verification code is: ${otp}. Valid for 10 minutes. Do
not share this code.
});

  return {
    success: true,
    message: 'OTP sent successfully'
  };

} catch (error) {
console.error('SMS sending error:', error);
return {
success: false,
message: 'Failed to send OTP'
};
}
};
```

```
module.exports = sendOTP;
```

Phone Validator

Create `backend/src/utils/validatePhone.js`:

```
const validatePhone = (phone) => {
// Nepal phone number validation
const nepalPhoneRegex = /^(98|97)[0-9]{8}$/;
return nepalPhoneRegex.test(phone);
};

module.exports = validatePhone;
```

Distance Calculator

Create `backend/src/utils/calculateDistance.js`:

```
// Calculate distance between two coordinates using Haversine
formula
const calculateDistance = (lat1, lon1, lat2, lon2) => {
const R = 6371; // Earth's radius in kilometers
const dLat = toRad(lat2 - lat1);
const dLon = toRad(lon2 - lon1);

const a =
Math.sin(dLat / 2) * Math.sin(dLat / 2) +
Math.cos(toRad(lat1)) * Math.cos(toRad(lat2)) *
Math.sin(dLon / 2) * Math.sin(dLon / 2);

const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
const distance = R * c;

return distance;
};

const toRad = (degrees) => {
return degrees * (Math.PI / 180);
};

module.exports = calculateDistance;
```

5.4 Error Handler Middleware

Create backend/src/middleware/errorHandler.js:

```
const errorHandler = (err, req, res, next) => {
let error = { ...err };
error.message = err.message;

// Log error for debugging
console.error('Error:', err);

// Mongoose bad ObjectId
if (err.name === 'CastError') {
const message = 'Resource not found';
error = { message, statusCode: 404 };
}

// Mongoose duplicate key
if (err.code === 11000) {
const field = Object.keys(err.keyValue)[0];
const message = ${field} already exists;
error = { message, statusCode: 400 };
}

// Mongoose validation error
if (err.name === 'ValidationError') {
const message = Object.values(err.errors).map(val =>
val.message).join(', ');
error = { message, statusCode: 400 };
}

res.status(error.statusCode || 500).json({
success: false,
message: error.message || 'Server Error',
...(process.env.NODE_ENV === 'development' && { stack: err.stack })
});
};

module.exports = errorHandler;
```

5.5 File Upload Middleware

Create backend/src/middleware/upload.js:

```
const multer = require('multer');
const cloudinary = require('cloudinary').v2;
const { CloudinaryStorage } = require('multer-storage-cloudinary');

// Configure Cloudinary
cloudinary.config({
cloud_name: process.env.CLOUDINARY_CLOUD_NAME,
api_key: process.env.CLOUDINARY_API_KEY,
api_secret: process.env.CLOUDINARY_API_SECRET
});

// Cloudinary storage
const storage = new CloudinaryStorage({
cloudinary: cloudinary,
params: {
folder: 'doko',
allowed_formats: ['jpg', 'jpeg', 'png', 'webp'],
transformation: [{ width: 800, height: 800, crop: 'limit' }]
}
});

const upload = multer({
storage: storage,
limits: {
fileSize: 5 * 1024 * 1024 // 5MB limit
},
fileFilter: (req, file, cb) => {
if (file.mimetype.startsWith('image/')) {
cb(null, true);
} else {
cb(new Error('Only image files are allowed'), false);
}
}
});

module.exports = upload;
```

5.6 Controllers Implementation

Auth Controller

Create `backend/src/controllers/authController.js`:

const User = require('../models/User');
const Vendor = require('../models/Vendor');
const Rider = require('../models/Rider');
const Admin = require('../models/Admin');
const generateToken = require('../utils/generateToken');
const sendOTP = require('../utils/sendOTP');
const validatePhone = require('../utils/validatePhone');

// @desc User Registration - Send OTP
// @route POST /api/v1/auth/user/register
// @access Public
exports.registerUser = async (req, res, next) => {
try {
const { phone, email, name } = req.body;

```
// Validate phone
if (!validatePhone(phone)) {
  return res.status(400).json({
    success: false,
    message: 'Invalid phone number format'
  });
}

// Check if user already exists
let user = await User.findOne({ $or: [{ phone }, { email }] });

if (user) {
  if (user.suspended) {
    return res.status(403).json({
      success: false,
      message: 'Account has been suspended',
      violations: user.violations
    });
```

```
    }

    return res.status(400).json({
      success: false,
      message: 'User already registered with this phone or email'
    });
  }

  // Create new user (without saving)
  user = new User({
    name,
    phone,
    email,
    passwordHash: Math.random().toString(36)  // Temporary, will be set later
  });

  // Generate and send OTP
  const otp = user.generateOTP();
  await user.save();

  const otpSent = await sendOTP(phone, otp);

  if (!otpSent.success) {
    return res.status(500).json({
      success: false,
      message: 'Failed to send OTP. Please try again.'
    });
  }

  res.status(200).json({
    success: true,
    message: 'OTP sent successfully to your phone'
  });
```

```
} catch (error) {
next(error);
}
};
```

// @desc Verify OTP and Complete Registration
// @route POST /api/v1/auth/user/verify-otp
// @access Public
exports.verifyUserOTP = async (req, res, next) => {
try {
const { phone, otp, password } = req.body;

```
const user = await User.findOne({ phone }).select('+passwordHash');

if (!user) {
  return res.status(404).json({
    success: false,
    message: 'User not found'
  });
}

if (!user.verifyOTP(otp)) {
  return res.status(400).json({
    success: false,
    message: 'Invalid or expired OTP'
  });
}

// Set password
user.passwordHash = password;
user.otp = undefined;
await user.save();

// Generate token
const token = generateToken(user._id, 'user');

res.status(200).json({
  success: true,
  message: 'Registration successful',
  token,
  user: {
    id: user._id,
    name: user.name,
```

```
    phone: user.phone,
    email: user.email
  }
});
```

} catch (error) {
next(error);
}
};

// @desc User Login - Send OTP
// @route POST /api/v1/auth/user/login
// @access Public
exports.loginUser = async (req, res, next) => {
try {
const { phone } = req.body;

```
const user = await User.findOne({ phone });

if (!user) {
  return res.status(404).json({
    success: false,
    message: 'User not found. Please register first.'
  });
}

if (user.suspended) {
  return res.status(403).json({
    success: false,
    message: 'Account has been suspended',
    violations: user.violations
  });
}

// Generate and send OTP
const otp = user.generateOTP();
await user.save();
```

```
  await sendOTP(phone, otp);

  res.status(200).json({
    success: true,
    message: 'OTP sent to your phone'
  });
```

} catch (error) {
next(error);
}
};

// @desc Verify Login OTP
// @route POST /api/v1/auth/user/login/verify-otp
// @access Public
exports.verifyLoginOTP = async (req, res, next) => {
try {
const { phone, otp } = req.body;

```
  const user = await User.findOne({ phone });

  if (!user || !user.verifyOTP(otp)) {
    return res.status(400).json({
      success: false,
      message: 'Invalid or expired OTP'
    });
  }

  user.otp = undefined;
  await user.save();

  const token = generateToken(user._id, 'user');

  res.status(200).json({
    success: true,
    message: 'Login successful',
    token,
    user: {
```

```
      id: user._id,
      name: user.name,
      phone: user.phone,
      email: user.email
    }
  });
```

} catch (error) {
next(error);
}
};

// Similar implementations for Vendor, Rider, Admin authentication...
// (Follow same pattern with role-specific models)

Product Controller

Create `backend/src/controllers/productController.js`:

const Product = require('../models/Product');
const Category = require('../models/Category');

// @desc Get all products with filters
// @route GET /api/v1/product/all
// @access Public
exports.getAllProducts = async (req, res, next) => {
try {
const {
page = 1,
limit = 20,
minPrice,
maxPrice,
productCategory,
deliveryCategory,
vendorId,
search,
stock,
brand,
rating
} = req.query;

```
// Build query
let query = { approved: true, hidden: false };

if (minPrice || maxPrice) {
  query.pricePerUnit = {};
  if (minPrice) query.pricePerUnit.$gte = Number(minPrice);
  if (maxPrice) query.pricePerUnit.$lte = Number(maxPrice);
}

if (productCategory) {
  query.productCategory = productCategory;
}

if (deliveryCategory) {
  query.deliveryCategory = deliveryCategory;
}

if (vendorId) {
  query.vendorId = vendorId;
}

if (brand) {
  query.brand = new RegExp(brand, 'i');
}

if (stock === 'inStock') {
  query.stock = { $gt: 0 };
}

if (rating) {
  query['rating.average'] = { $gte: Number(rating) };
}

if (search) {
  query.$text = { $search: search };
}
```

```
// Execute query with pagination
const products = await Product.find(query)
  .populate('productCategory', 'name type')
  .populate('vendorId', 'storeName')
  .limit(Number(limit))
  .skip((Number(page) - 1) * Number(limit))
  .sort('-createdAt');

const total = await Product.countDocuments(query);

res.status(200).json({
  success: true,
  count: products.length,
  total,
  page: Number(page),
  pages: Math.ceil(total / limit),
  data: products
});
```

} catch (error) {
next(error);
}
};

// @desc Get single product by ID
// @route GET /api/v1/product/:id
// @access Public
exports.getProductById = async (req, res, next) => {
try {
const product = await Product.findById(req.params.id)
.populate('productCategory', 'name type')
.populate('vendorId', 'storeName address geolocation');

```
if (!product) {
  return res.status(404).json({
    success: false,
    message: 'Product not found'
  });
```

```
  }

  res.status(200).json({
    success: true,
    data: product
  });
```

} catch (error) {
next(error);
}
};

// @desc Create new product (Vendor only)
// @route POST /api/v1/product
// @access Private/Vendor
exports.createProduct = async (req, res, next) => {
try {
// Add vendor ID from authenticated user
req.body.vendorId = req.user._id;

```
  // Set approved to false by default (awaiting admin approval)
  req.body.approved = false;

  const product = await Product.create(req.body);

  res.status(201).json({
    success: true,
    message: 'Product submitted for approval',
    data: product
  });
```

} catch (error) {
next(error);
}
};

// @desc Update product
// @route PUT /api/v1/product/:id

// @access Private/Vendor
exports.updateProduct = async (req, res, next) => {
try {
let product = await Product.findById(req.params.id);

```
if (!product) {
  return res.status(404).json({
    success: false,
    message: 'Product not found'
  });
}

// Check ownership
if (product.vendorId.toString() !== req.user._id.toString()) {
  return res.status(403).json({
    success: false,
    message: 'Not authorized to update this product'
  });
}

// Set approved to false (needs re-approval after edit)
req.body.approved = false;

product = await Product.findByIdAndUpdate(req.params.id, req.body, {
  new: true,
  runValidators: true
});

res.status(200).json({
  success: true,
  message: 'Product updated and submitted for re-approval',
  data: product
});
```

} catch (error) {
next(error);

}
};

// @desc Delete product
// @route DELETE /api/v1/product/:id
// @access Private/Vendor
exports.deleteProduct = async (req, res, next) => {
try {
const product = await Product.findById(req.params.id);

```
if (!product) {
  return res.status(404).json({
    success: false,
    message: 'Product not found'
  });
}

// Check ownership
if (product.vendorId.toString() !== req.user._id.toString()) {
  return res.status(403).json({
    success: false,
    message: 'Not authorized to delete this product'
  });
}

await product.deleteOne();

res.status(200).json({
  success: true,
  message: 'Product deleted successfully'
});
```

} catch (error) {
next(error);
}
};

5.7 Routes Setup

Auth Routes

Create backend/src/routes/auth.js:

```
const express = require('express');
const router = express.Router();
const {
registerUser,
verifyUserOTP,
loginUser,
verifyLoginOTP
} = require('../controllers/authController');

// User authentication
router.post('/user/register', registerUser);
router.post('/user/verify-otp', verifyUserOTP);
router.post('/user/login', loginUser);
router.post('/user/login/verify-otp', verifyLoginOTP);

// Similar routes for vendor, rider, admin...

module.exports = router;
```

Product Routes

Create backend/src/routes/product.js:

```
const express = require('express');
const router = express.Router();
const {
getAllProducts,
getProductById,
createProduct,
updateProduct,
deleteProduct
} = require('../controllers/productController');
const { protect, isVendor } = require('../middleware/auth');

// Public routes
router.get('/all', getAllProducts);
```

```
router.get('/:id', getProductById);

// Vendor routes
router.post('/', protect, isVendor, createProduct);
router.put('/:id', protect, isVendor, updateProduct);
router.delete('/:id', protect, isVendor, deleteProduct);

module.exports = router;
```

---

6. Frontend Application Development

6.1 Initialize React Project

```
# Navigate to project root
cd doko-platform

# Create React app
npx create-react-app frontend

# Navigate to frontend
cd frontend

# Install dependencies
npm install react-router-dom axios
npm install react-leaflet leaflet
npm install socket.io-client
npm install react-toastify
npm install @headlessui/react @heroicons/react
```

Start development server

npm start

6.2 Environment Configuration

Create frontend/.env:

```
REACT_APP_API_URL=http://localhost:5000/api/v1
REACT_APP_SOCKET_URL=http://localhost:5000
REACT_APP_GOOGLE_MAPS_KEY=your_google_maps_api_key
REACT_APP_KHALTI_PUBLIC_KEY=your_khalti_public_key
```

6.3 API Service Setup

Create frontend/src/services/api.js:

```
import axios from 'axios';

const API_BASE_URL = process.env.REACT_APP_API_URL || '
http://localhost:5000/api/v1';

const api = axios.create({
baseURL: API_BASE_URL,
headers: {
'Content-Type': 'application/json'
}
});

// Add auth token to requests
api.interceptors.request.use(
(config) => {
const token = localStorage.getItem('token');
if (token) {
config.headers.Authorization = Bearer ${token};
}
return config;
},
(error) => Promise.reject(error)
);
```

```
// Handle errors globally
api.interceptors.response.use(
(response) => response,
(error) => {
if (error.response?.status === 401) {
localStorage.removeItem('token');
window.location.href = '/login';
}
return Promise.reject(error);
}
);

// Auth APIs
export const authAPI = {
register: (data) => api.post('/auth/user/register', data),
verifyOTP: (data) => api.post('/auth/user/verify-otp', data),
login: (data) => api.post('/auth/user/login', data),
verifyLoginOTP: (data) => api.post('/auth/user/login/verify-otp', data)
};

// Product APIs
export const productAPI = {
getAll: (params) => api.get('/product/all', { params }),
getById: (id) => api.get(/product/${id}),
create: (data) => api.post('/product', data),
update: (id, data) => api.put(/product/${id}, data),
delete: (id) => api.delete(/product/${id})
};

// Cart APIs
export const cartAPI = {
get: () => api.get('/cart'),
add: (data) => api.post('/cart/add', data),
update: (id, data) => api.put(/cart/${id}, data),
remove: (id) => api.delete(/cart/${id}),
clear: () => api.delete('/cart/clear')
};

// Order APIs
export const orderAPI = {
```

```
create: (data) => api.post('/order', data),
getAll: (params) => api.get('/order/all', { params }),
getById: (id) => api.get(/order/${id}),
cancel: (id) => api.post(/order/${id}/cancel)
};

export default api;
```

6.4 Context Setup

Auth Context

Create `frontend/src/context/AuthContext.jsx`:

```
import React, { createContext, useState, useEffect } from 'react';
import { authAPI } from '../services/api';

export const AuthContext = createContext();

export const AuthProvider = ({ children }) => {
const [user, setUser] = useState(null);
const [token, setToken] = useState(localStorage.getItem('token'));
const [loading, setLoading] = useState(true);

useEffect(() => {
if (token) {
// Fetch user data
fetchUser();
} else {
setLoading(false);
}
}, [token]);

const fetchUser = async () => {
try {
const { data } = await api.get('/user/profile');
setUser(data.user);
} catch (error) {
console.error('Failed to fetch user:', error);
logout();
} finally {
setLoading(false);
```

```
}
};

const login = (newToken, userData) => {
localStorage.setItem('token', newToken);
setToken(newToken);
setUser(userData);
};

const logout = () => {
localStorage.removeItem('token');
setToken(null);
setUser(null);
};

return (
<AuthContext.Provider value={{ user, token, loading, login, logout }}>
{children}
</AuthContext.Provider>
);
};
```

Cart Context

Create `frontend/src/context/CartContext.jsx`:

```
import React, { createContext, useState, useEffect } from 'react';
import { cartAPI } from '../services/api';

export const CartContext = createContext();

export const CartProvider = ({ children }) => {
const [cart, setCart] = useState({ items: [], total: 0 });
const [loading, setLoading] = useState(false);

useEffect(() => {
fetchCart();
}, []);

const fetchCart = async () => {
try {
setLoading(true);
```

```
const { data } = await cartAPI.get();
setCart(data.cart);
} catch (error) {
console.error('Failed to fetch cart:', error);
} finally {
setLoading(false);
}
};

const addToCart = async (productId, quantity = 1) => {
try {
const { data } = await cartAPI.add({ productId, quantity });
setCart(data.cart);
return { success: true };
} catch (error) {
return { success: false, message: error.response?.data?.message };
}
};

const removeFromCart = async (cartItemId) => {
try {
await cartAPI.remove(cartItemId);
fetchCart();
} catch (error) {
console.error('Failed to remove item:', error);
}
};

const clearCart = async () => {
try {
await cartAPI.clear();
setCart({ items: [], total: 0 });
} catch (error) {
console.error('Failed to clear cart:', error);
}
};

return (
<CartContext.Provider value={{
cart,
```

```
loading,
addToCart,
removeFromCart,
clearCart,
refreshCart: fetchCart
}}>
{children}
</CartContext.Provider>
);
};
```

6.5 Component Implementation

Navbar Component

Create frontend/src/components/common/Navbar.jsx:

```
import React, { useState, useContext } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { AuthContext } from '../../context/AuthContext';
import { CartContext } from '../../context/CartContext';

const Navbar = () => {
const { user, logout } = useContext(AuthContext);
const { cart } = useContext(CartContext);
const [searchQuery, setSearchQuery] = useState('');
const navigate = useNavigate();

const handleSearch = (e) => {
e.preventDefault();
if (searchQuery.trim()) {
navigate(/products?search=${encodeURIComponent(searchQuery)});
}
};

return (
<nav className="bg-white shadow-md sticky top-0 z-50">
<div className="container mx-auto px-4">
<div className="flex items-center justify-between h-16">
{/* Logo */}
```

Doko
</Link>

```
{/* Search Bar */}
<form onSubmit={handleSearch} className="flex-1 max-w-2xl mx-8">
  <div className="relative">
    <input
      type="text"
      placeholder="Search for products..."
      value={searchQuery}
      onChange={(e) => setSearchQuery(e.target.value)}
      className="w-full px-4 py-2 pr-10 border border-gray-300 rounded-lg
    />
    <button type="submit" className="absolute right-3 top-2.5">
      
    </button>
  </div>
</form>

{/* Navigation Links */}
<div className="flex items-center space-x-6">
  <Link to="/products" className="text-gray-700 hover:text-green-600">
    Products
  </Link>

  <Link to="/cart" className="relative text-gray-700 hover:text-green-600"
    Cart
    {cart.items?.length > 0 && (
      <span className="absolute -top-2 -right-2 bg-red-500 text-white text-x
        {cart.items.length}
      </span>
    )}
  </Link>

  <Link to="/wishlist" className="text-gray-700 hover:text-green-600">
    ❤ Wishlist
  </Link>
```

```
        {user ? (
          <div className="flex items-center space-x-4">
            <Link to="/orders" className="text-gray-700 hover:text-green-600">
              Orders
            </Link>
            <Link to="/profile" className="text-gray-700 hover:text-green-600">
              Profile
            </Link>
            <button
              onClick={logout}
              className="bg-red-500 text-white px-4 py-2 rounded-lg hover:bg-red
            >
              Logout
            </button>
          </div>
        ) : (
          <Link
            to="/login"
            className="bg-green-600 text-white px-6 py-2 rounded-lg hover:bg-gr
          >
            Login / Sign Up
          </Link>
        )}
      </div>
    </div>
  </div>
</nav>
```

);
};

export default Navbar;

Product Card Component

Create `frontend/src/components/product/ProductCard.jsx`:

```
import React, { useContext } from 'react';
import { Link } from 'react-router-dom';
import { CartContext } from '../../context/CartContext';
import { toast } from 'react-toastify';

const ProductCard = ({ product }) => {
const { addToCart } = useContext(CartContext);

const handleAddToCart = async () => {
const result = await addToCart(product._id);
if (result.success) {
toast.success('Added to cart!');
} else {
toast.error(result.message || 'Failed to add to cart');
}
};

const finalPrice = product.pricePerUnit - (product.pricePerUnit *
product.discount / 100);

return (
<div className="bg-white rounded-lg shadow-md overflow-hidden
hover:shadow-xl transition-shadow">
<Link to={/product/${product._id}}>

</Link>
```

```
<div className="p-4">
  <Link to={`/product/${product._id}`}>
    <h3 className="text-lg font-semibold text-gray-800 hover:text-green-600
      {product.name}
    </h3>
  </Link>

  <p className="text-sm text-gray-600 mt-1 truncate">
    {product.brand}
  </p>

  <div className="mt-2 flex items-center justify-between">
```

```
      <div>
        <span className="text-xl font-bold text-gray-900">
          NPR {finalPrice.toFixed(2)}
        </span>
        {product.discount > 0 && (
          <div className="flex items-center space-x-2 mt-1">
            <span className="text-sm text-gray-500 line-through">
              NPR {product.pricePerUnit}
            </span>
            <span className="text-xs bg-red-500 text-white px-2 py-0.5 rounded">
              {product.discount}% OFF
            </span>
          </div>
        )}
      </div>

      <div className="flex items-center">
        <span className="text-yellow-500">★</span>
        <span className="text-sm text-gray-600 ml-1">
          {product.rating?.average?.toFixed(1) || 'N/A'}
        </span>
      </div>
    </div>

    <button
      onClick={handleAddToCart}
      disabled={product.stock === 0}
      className={`mt-4 w-full py-2 px-4 rounded-lg font-semibold transition-co
        product.stock === 0
          ? 'bg-gray-300 text-gray-500 cursor-not-allowed'
          : 'bg-green-600 text-white hover:bg-green-700'
      }`}
    >
      {product.stock === 0 ? 'Out of Stock' : 'Add to Cart'}
    </button>
  </div>
</div>
```

```
);
};

export default ProductCard;
```

6.6 Page Implementation

Home Page

Create `frontend/src/pages/customer/Home.jsx`:

```
import React, { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';
import { productAPI } from '../../services/api';
import ProductCard from '../../components/product/ProductCard';

const Home = () => {
const [featuredProducts, setFeaturedProducts] = useState([]);
const [categories, setCategories] = useState([]);
const [loading, setLoading] = useState(true);

useEffect(() => {
fetchHomeData();
}, []);

const fetchHomeData = async () => {
try {
setLoading(true);
```

```
// Fetch featured products
const { data } = await productAPI.getAll({
  limit: 8,
  deliveryCategory: 'quick'
});
setFeaturedProducts(data.data);

// Fetch categories (mock for now)
setCategories([
  { name: 'Groceries', icon: '', slug: 'groceries' },
  { name: 'Vegetables', icon: '', slug: 'vegetables' },
  { name: 'Fruits', icon: '', slug: 'fruits' },
```

```
    { name: 'Dairy', icon: '', slug: 'dairy' },
    { name: 'Bakery', icon: '', slug: 'bakery' },
    { name: 'Beverages', icon: '', slug: 'beverages' }
  ]);

} catch (error) {
  console.error('Failed to fetch home data:', error);
} finally {
  setLoading(false);
}
```

};

if (loading) {
return
Loading...
;
}

return (

{/* Hero Section */}

Quick Commerce, Delivered Fast

Get your groceries and essentials delivered in minutes

Shop Now
</Link>

```
  {/* Categories */}
  <section className="py-12">
    <div className="container mx-auto px-4">
      <h2 className="text-3xl font-bold mb-8">Shop by Category</h2>
```

```
    <div className="grid grid-cols-2 md:grid-cols-3 lg:grid-cols-6 gap-4">
      {categories.map((category) => (
        <Link
          key={category.slug}
          to={`/products?category=${category.slug}`}
          className="bg-white p-6 rounded-lg shadow-md hover:shadow-xl tra
        >
          <div className="text-5xl mb-2">{category.icon}</div>
          <h3 className="text-lg font-semibold">{category.name}</h3>
        </Link>
      ))}
    </div>
  </div>
</section>

{/* Featured Products */}
<section className="py-12 bg-white">
  <div className="container mx-auto px-4">
    <h2 className="text-3xl font-bold mb-8">Featured Products</h2>
    <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-6">
      {featuredProducts.map((product) => (
        <ProductCard key={product._id} product={product} />
      ))}
    </div>
  </div>
</section>

{/* Why Choose Us */}
<section className="py-12">
  <div className="container mx-auto px-4">
    <h2 className="text-3xl font-bold mb-8 text-center">Why Choose Doko?<
    <div className="grid grid-cols-1 md:grid-cols-3 gap-8">
      <div className="text-center">
        <div className="text-6xl mb-4">⚡</div>
        <h3 className="text-xl font-semibold mb-2">Lightning Fast Delivery</h
        <p className="text-gray-600">Get your orders delivered in 10-30 minu
      </div>
      <div className="text-center">
```

```
          <div className="text-6xl mb-4">✓ </div>
          <h3 className="text-xl font-semibold mb-2">Fresh & Quality Products
          <p className="text-gray-600">Hand-picked fresh products every time</
        </div>
        <div className="text-center">
          <div className="text-6xl mb-4">□</div>
          <h3 className="text-xl font-semibold mb-2">Easy Payments</h3>
          <p className="text-gray-600">Multiple payment options available</p>
        </div>
      </div>
    </div>
  </section>
</div>
```

);
};

export default Home;

---

7. Payment Integration

7.1 Khalti Integration

Create `backend/src/services/khalti.js`:

```
const axios = require('axios');

const KHALTI_SECRET_KEY = process.env.KHALTI_SECRET_KEY;
const KHALTI_API = 'https://khalti.com/api/v2';

exports.initiatePayment = async (orderData) => {
try {
const response = await axios.post(
${KHALTI_API}/payment/initiate/,
{
return_url: ${process.env.FRONTEND_URL}/payment/callback,
website_url: process.env.FRONTEND_URL,
amount: orderData.amount * 100, // Convert to paisa
purchase_order_id: orderData.orderId,
```

```
purchase_order_name: 'Doko Order',
customer_info: {
name: orderData.customerName,
email: orderData.customerEmail,
phone: orderData.customerPhone
}
},
{
headers: {
Authorization: Key ${KHALTI_SECRET_KEY}
}
}
);
```

```
return {
  success: true,
  paymentUrl: response.data.payment_url,
  pidx: response.data.pidx
};
```

```
} catch (error) {
console.error('Khalti payment initiation error:', error);
return {
success: false,
message: 'Failed to initiate payment'
};
}
};

exports.verifyPayment = async (pidx) => {
try {
const response = await axios.post(
${KHALTI_API}/payment/verify/,
{ pidx },
{
headers: {
Authorization: Key ${KHALTI_SECRET_KEY}
}
```

}
);

```
return {
  success: true,
  data: response.data
};
```

} catch (error) {
return {
success: false,
message: 'Payment verification failed'
};
}
};

7.2 eSewa Integration

Create `backend/src/services/esewa.js`:

```
const crypto = require('crypto');

const ESEWA_MERCHANT_ID = process.env.ESEWA_MERCHANT_ID;
const ESEWA_SECRET_KEY = process.env.ESEWA_SECRET_KEY;
const ESEWA_URL = 'https://esewa.com.np/epay/main';

exports.generateSignature = (data) => {
const message =
total_amount=${data.amount},transaction_uuid=${data.transactionId},product_code=${ESEWA_MERCHANT_ID};

const hmac = crypto.createHmac('sha256', ESEWA_SECRET_KEY);
hmac.update(message);

return hmac.digest('base64');
};

exports.initiatePayment = (orderData) => {
const transactionId = DOKO-${Date.now()};
```

```
const signature = exports.generateSignature({
amount: orderData.amount,
transactionId
});

return {
success: true,
url: ESEWA_URL,
data: {
amount: orderData.amount,
tax_amount: 0,
total_amount: orderData.amount,
transaction_uuid: transactionId,
product_code: ESEWA_MERCHANT_ID,
product_service_charge: 0,
product_delivery_charge: 0,
success_url: ${process.env.FRONTEND_URL}/payment/esewa/success,
failure_url: ${process.env.FRONTEND_URL}/payment/esewa/failure,
signed_field_names: 'total_amount,transaction_uuid,product_code',
signature
}
};
};
```

---

8. Real-time Features with Socket.io

8.1 Socket Server Setup

Create backend/src/services/socket.js:

```
const socketIO = require('socket.io');
const jwt = require('jsonwebtoken');

let io;

exports.initializeSocket = (server) => {
io = socketIO(server, {
cors: {
origin: process.env.FRONTEND_URL,
credentials: true
```

}
});

// Authentication middleware
io.use(async (socket, next) => {
try {
const token = socket.handshake.auth.token;
if (!token) {
return next(new Error('Authentication error'));
}

```
  const decoded = jwt.verify(token, process.env.JWT_SECRET);
  socket.userId = decoded.id;
  socket.userRole = decoded.role;
  next();
} catch (error) {
  next(new Error('Authentication error'));
}
```

});

io.on('connection', (socket) => {
console.log(`✅ User connected: ${socket.userId} (${socket.userRole})`);

```
// Join user-specific room
socket.join(`user:${socket.userId}`);

// Rider location updates
if (socket.userRole === 'rider') {
  socket.on('updateLocation', (location) => {
    socket.broadcast.emit('riderLocation', {
      riderId: socket.userId,
      location
    });
  });
}

// Order status updates
```

```
  socket.on('orderUpdate', (data) => {
    io.to(`user:${data.userId}`).emit('orderStatusChanged', data);
  });

  socket.on('disconnect', () => {
    console.log(`✕ User disconnected: ${socket.userId}`);
  });
```

```
});

return io;
};

exports.getIO = () => {
if (!io) {
throw new Error('Socket.io not initialized');
}
return io;
};

exports.emitToUser = (userId, event, data) => {
io.to(user:${userId}).emit(event, data);
};

exports.emitToRider = (riderId, event, data) => {
io.to(user:${riderId}).emit(event, data);
};
```

Update backend/src/server.js to initialize Socket.io:

```
// ... existing code ...

const http = require('http');
const { initializeSocket } = require('./services/socket');

// Create HTTP server
const server = http.createServer(app);

// Initialize Socket.io
initializeSocket(server);
```

```
// Start server
server.listen(PORT, () => {
// ... existing console log ...
});
```

8.2 Frontend Socket Integration

Create `frontend/src/services/socket.js`:

```
import { io } from 'socket.io-client';

let socket;

export const initializeSocket = (token) => {
socket = io(process.env.REACT_APP_SOCKET_URL, {
auth: { token }
});

socket.on('connect', () => {
console.log('✅ Connected to server');
});

socket.on('disconnect', () => {
console.log('❌ Disconnected from server');
});

return socket;
};

export const getSocket = () => {
if (!socket) {
throw new Error('Socket not initialized');
}
return socket;
};

export const disconnectSocket = () => {
if (socket) {
socket.disconnect();
}
};
```

---

9. Testing Strategy

9.1 Backend API Testing

Create `backend/tests/product.test.js`:

const request = require('supertest');
const app = require('../src/server');
const Product = require('../src/models/Product');

describe('Product API Tests', () => {
let authToken;
let productId;

beforeAll(async () => {
// Login and get token
const res = await request(app)
.post('/api/v1/auth/user/login/verify-otp')
.send({ phone: '9800000000', otp: '123456' });

```
authToken = res.body.token;
```

});

test('GET /api/v1/product/all - should return products', async () => {
const res = await request(app)
.get('/api/v1/product/all')
.expect(200);

```
expect(res.body.success).toBe(true);
expect(Array.isArray(res.body.data)).toBe(true);
```

});

test('POST /api/v1/product - should create product (vendor)', async () => {
const newProduct = {
name: 'Test Product',
brand: 'Test Brand',

shortDescription: 'Short desc',
description: 'Long description',
pricePerUnit: 100,
unit: 'pcs',
productCategory: 'category_id_here',
deliveryCategory: 'quick',
stock: 50,
photos: ['https://example.com/photo.jpg']
};

```
const res = await request(app)
  .post('/api/v1/product')
  .set('Authorization', `Bearer ${authToken}`)
  .send(newProduct)
  .expect(201);

expect(res.body.success).toBe(true);
productId = res.body.data._id;
```

});

afterAll(async () => {
// Cleanup: Delete test product
if (productId) {
await Product.findByIdAndDelete(productId);
}
});
});

Run tests:
npm test

---

10. Deployment Guide

10.1 Backend Deployment (Railway)

Step 1: Prepare for deployment

Create `backend/.dockerignore`:
node_modules
npm-debug.log
.env
.git

Step 2: Deploy to Railway

Install Railway CLI

npm install -g @railway/cli

Login

railway login

Initialize project

cd backend
railway init

Add environment variables in Railway dashboard

Deploy

railway up

10.2 Frontend Deployment (Vercel)

Install Vercel CLI

npm install -g vercel

Navigate to frontend

cd frontend

Deploy

vercel

Follow prompts and set environment variables

10.3 Database (MongoDB Atlas)

1. Create account at https://www.mongodb.com/cloud/atlas
2. Create new cluster (M0 Free tier)
3. Add database user
4. Whitelist IP addresses (0.0.0.0/0 for all)
5. Get connection string
6. Update MONGODB_URI in backend environment variables

---

11. Appendix: Complete API Reference

Authentication Endpoints

Endpoint	Method	Description
/api/v1/auth/user/register	POST	Register new user
/api/v1/auth/user/verify-otp	POST	Verify OTP
/api/v1/auth/user/login	POST	Login user
/api/v1/auth/user/login/verify-otp	POST	Verify login OTP
/api/v1/auth/vendor/register	POST	Register vendor
/api/v1/auth/rider/register	POST	Register rider

Table 4: Authentication API endpoints

Product Endpoints

Endpoint	Method	Auth
/api/v1/product/all	GET	Public
/api/v1/product/:id	GET	Public
/api/v1/product	POST	Vendor
/api/v1/product/:id	PUT	Vendor
/api/v1/product/:id	DELETE	Vendor

Table 5: Product API endpoints

(Complete API reference continues in separate documentation file)

References

[1] MongoDB Inc. (2024). MongoDB Atlas Documentation. https://www.mongodb.com/docs/atlas/

[2] Meta Platforms. (2024). React Documentation. https://react.dev/

[3] Vercel Inc. (2024). Next.js Documentation - Project Structure Best Practices. https://nextjs.org/docs

[4] GeeksforGeeks. (2024). How to Design a Database for 10-minute Grocery Delivery App like Zepto. https://www.geeksforgeeks.org/dbms/how-to-design-a-database-for-10-minute-grocery-delivery-app-like-zepto/

[5] LinkedIn. (2026). Designing Foodela's Scalable Database Schema for Food Delivery. https://www.linkedin.com/posts/semmozhiyan-n-s-aa7478296_erdiagram-databasedesign-systemdesign-activity-7426154758979371009-flPS

---

1. 0-9 ↩
2. 0-9 ↩
3. 0-9 ↩